МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 5

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

Отчет представлен «26» сентября 2026 г.

Москва 2026 г.

ЗАДАНИЕ 1. Разработка пользовательских функций

1.1 Функция, возвращающая скалярное значение

```sql
-- Функция возвращает общую сумму всех покупок клиента
CREATE OR REPLACE FUNCTION get_client_total_spending(p_client_id INT)
RETURNS DECIMAL(12, 2) AS $$
DECLARE
    v_total_amount DECIMAL(12, 2);
BEGIN
    -- Считаем общую сумму продаж
    SELECT COALESCE(SUM(s.total_amount), 0.00)
    INTO v_total_amount
    FROM sale s
    WHERE s.client_id = p_client_id;

    RETURN v_total_amount;
END;
$$ LANGUAGE plpgsql;

-- Вызов функции
SELECT
    id,
    (full_name).last_name || ' ' || (full_name).first_name AS "Клиент",
    get_client_total_spending(id) AS "Общая сумма покупок"
FROM client
ORDER BY "Общая сумма покупок" DESC;
```

client 1 | Статистика 1

Таблица | Текст

	id	Клиент	Общая сумма покупок
1	6	Морозова Елена	1 250
2	1	Иванов Алексей	950
3	9	Соколов Андрей	920
4	4	Козлова Анна	890
5	8	Лебедева Ольга	780
6	5	Новиков Игорь	670
7	3	Сидоров Дмитрий	580
8	10	Павлова Татьяна	450
9	7	Волков Сергей	340
10	2	Петрова Мария	320
11	11	[NULL]	0

Функция принимает p_client_id и возвращает одно число (DECIMAL)

Проверяет существование клиента через EXISTS

Считает сумму через SUM(total_amount) из таблицы sale

COALESCE заменяет NULL на 0.00 (если у клиента нет покупок)

Вызывается как часть SELECT для каждого клиента

1.2 Функция с OUT-параметрами (возврат нескольких значений)

Задача: Создать функцию, которая по ID товара возвращает его название, текущий остаток на складе и цену.

```
CREATE OR REPLACE FUNCTION get_product_info(
    p_product_id INT,
    OUT p_name VARCHAR(100),
    OUT p_total_quantity INT,       -- Общий остаток по всем складам
    OUT p_price DECIMAL(10, 2)
)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT
        pr.name,
        COALESCE(SUM(st.quantity), 0),  -- Суммируем остатки из всех магазинов
        pr.retail_price
    INTO p_name, p_total_quantity, p_price
    FROM product pr
    LEFT JOIN stock st ON pr.id = st.product_id
    WHERE pr.id = p_product_id
    GROUP BY pr.id, pr.name, pr.retail_price;

    -- Если товар не найден, переменная FOUND будет FALSE
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Товар с ID % не найден.', p_product_id;
    END IF;
END;
$$;

SELECT * FROM get_product_info(1);
```

Результат 1 | Статистика 1

Таблица

	A-Z p_name	123 p_total_quantity	123 p_price
1	Coca-Cola Classic 0.5л	335	85

OUT параметры — это «выходные» переменные, которые функция заполняет

PostgreSQL автоматически формирует из них составной тип (строку)

SELECT ... INTO помещает значения из запроса в переменные

FOUND — системная переменная: TRUE, если SELECT вернул строки

1.3 Функция, возвращающая таблицу (RETURNS TABLE)

Задача: Создать функцию, которая возвращает все товары определённой категории с их производителями.

```sql
-- Функция возвращает таблицу товаров категории
CREATE OR REPLACE FUNCTION get_products_by_category(p_category_id INT)
RETURNS TABLE (
    product_id INT,
    product_name VARCHAR(100),
    manufacturer_name VARCHAR(100),
    retail_price DECIMAL(10, 2),
    unit_name VARCHAR(30)
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.id,
        p.name,
        m.name,
        p.retail_price,
        u.name
    FROM product p
    JOIN manufacturer m ON p.manufacturer_id = m.id
    JOIN unit u ON p.unit_id = u.id
    WHERE p.category_id = p_category_id
    ORDER BY p.name;
END;
$$;

-- Вызов функции
SELECT * FROM get_products_by_category(1);  -- Категория 1 = Газированные напитки

-- Или с JOIN к другим таблицам
SELECT
    t.product_name AS "Товар",
    t.manufacturer_name AS "Производитель",
    t.retail_price AS "Цена"
FROM get_products_by_category(4) AS t  -- Категория 4 = Энергетики
WHERE t.retail_price > 100;
```

Результат 1 | Результат 1 (2) | Статистика 1

Таблица

	Товар	Производитель	Цена
1	Monster Energy 0.5л	Monster Energy	145
2	Red Bull 0.25л	Red Bull	150

RETURNS TABLE (...) объявляет структуру возвращаемой таблицы

RETURN QUERY выполняет запрос и возвращает его результат

ЗАДАНИЕ 2. Разработка хранимой процедуры с управлением транзакциями

Задача: Создать процедуру оформления продажи напитка с проверкой остатков и автоматическим управлением транзакцией.

2.1 Создание процедуры (Версия 1: Базовая логика)

```sql
-- Хранимая процедура оформления продажи
CREATE OR REPLACE PROCEDURE process_sale(
    p_product_id INT,           -- ID товара
    p_employee_id INT,          -- ID сотрудника
    p_client_id INT,            -- ID клиента (может быть NULL)
    p_quantity INT,             -- Количество товара
    p_payment_type_id INT,      -- Способ оплаты
    OUT p_success BOOLEAN,      -- Флаг успеха
    OUT p_message TEXT,         -- Сообщение
    OUT p_sale_id INT           -- ID созданной продажи
)
LANGUAGE plpgsql AS $$
DECLARE
    v_current_stock INT;
    v_price DECIMAL(10, 2);
    v_total_amount DECIMAL(12, 2);
    v_product_exists BOOLEAN;
BEGIN
    -- Инициализация выходных параметров
    p_success := FALSE;
    p_message := '';
    p_sale_id := NULL;

    -- Проверка существования товара
    SELECT EXISTS(SELECT 1 FROM product WHERE id = p_product_id)
    INTO v_product_exists;

    IF NOT v_product_exists THEN
        p_message := 'Ошибка: товар с ID ' || p_product_id || ' не найден.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Получаем текущий остаток и цену (с блокировкой строки)
    SELECT quantity, retail_price
    INTO v_current_stock, v_price
    FROM product
    WHERE id = p_product_id
    FOR UPDATE;  -- Блокируем строку для предотвращения гонки


-- Проверка достаточности товара
IF v_current_stock < p_quantity THEN
    p_message := 'Недостаточно товара на складе. В наличии: ' ||
                 v_current_stock || ', запрошено: ' || p_quantity;
    ROLLBACK;
    RETURN;
END IF;

-- Рассчитываем общую сумму
v_total_amount := v_price * p_quantity;

-- Списываем товар со склада
UPDATE product
SET quantity = quantity - p_quantity
WHERE id = p_product_id;

-- Создаём запись о продаже (шапка)
INSERT INTO sale (
    number,
    date,
    employee_id,
    client_id,
    payment_type_id,
    total_amount
) VALUES (
    'SALE-' || TO_CHAR(CURRENT_TIMESTAMP, 'YYYY-MM-DD-HH24-MI-SS'),
    CURRENT_TIMESTAMP,
    p_employee_id,
    p_client_id,
    p_payment_type_id,
    v_total_amount
)
RETURNING id INTO p_sale_id;
```

```
    -- Создаём позицию продажи
    INSERT INTO sale_item (
        sale_id,
        product_id,
        quantity,
        price
    ) VALUES (
        p_sale_id,
        p_product_id,
        p_quantity,
        v_price
    );

    -- Фиксируем транзакцию
    COMMIT;

    p_success := TRUE;
    p_message := 'Продажа успешно оформлена. ID продажи: ' || p_sale_id ||
                 '. Сумма: ' || v_total_amount || ' руб.';

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_success := FALSE;
        p_message := 'Системная ошибка: ' || SQLERRM;
END;
$$;
```

Проверка существования товара — через EXISTS

Блокировка строки — FOR UPDATE предотвращает «состояние гонки» (два кассира не могут одновременно продать последний товар)

Проверка остатка — если товара недостаточно, ROLLBACK и возврат ошибки

Списание товара — UPDATE product SET quantity = quantity - p_quantity

Создание продажи — два INSERT: в sale (шапка) и sale_item (позиция)

COMMIT — фиксируем все изменения

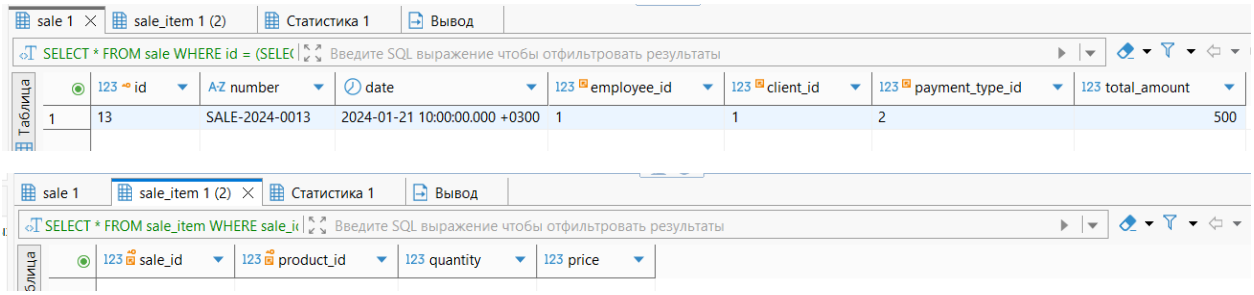
EXCEPTION — перехватывает любые ошибки и делает ROLLBACK

2.2 Демонстрация вызова процедуры

```
-- Тестовый вызов процедуры
DO $$
DECLARE
    v_success BOOLEAN;
    v_message TEXT;
    v_sale_id INT;
BEGIN
    CALL process_sale(
        p_product_id := 1,          -- Coca-Cola
        p_employee_id := 1,         -- Смирнов А.В.
        p_client_id := 1,           -- Иванов А.П.
        p_quantity := 2,            -- 2 штуки
        p_payment_type_id := 2,     -- Банковская карта
        p_success := v_success,
        p_message := v_message,
        p_sale_id := v_sale_id
    );

    -- Вывод результата
    RAISE NOTICE 'Результат: %', v_message;
    RAISE NOTICE 'Успех: %', v_success;
    RAISE NOTICE 'ID продажи: %', v_sale_id;
END;
$$;

-- Проверяем, что продажа создалась
SELECT * FROM sale WHERE id = (SELECT MAX(id) FROM sale);
SELECT * FROM sale_item WHERE sale_id = (SELECT MAX(id) FROM sale);
```



2.3 Финальная версия процедуры (без EXCEPTION, с явным управлением)

```sql
-- Финальная версия: убираем EXCEPTION, чтобы можно было использовать COMMIT
CREATE OR REPLACE PROCEDURE process_sale_final(
    p_product_id INT,
    p_employee_id INT,
    p_client_id INT,
    p_quantity INT,
    p_payment_type_id INT,
    OUT p_success BOOLEAN,
    OUT p_message TEXT,
    OUT p_sale_id INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_current_stock INT;
    v_price DECIMAL(10, 2);
    v_total_amount DECIMAL(12, 2);
    v_product_exists BOOLEAN;
    v_client_exists BOOLEAN;
BEGIN
    p_success := FALSE;
    p_message := '';
    p_sale_id := NULL;

    -- Проверка товара
    SELECT EXISTS(SELECT 1 FROM product WHERE id = p_product_id)
    INTO v_product_exists;

    IF NOT v_product_exists THEN
        p_message := 'Товар не найден';
        ROLLBACK;
        RETURN;
    END IF;

    -- Проверка клиента (если указан)
    IF p_client_id IS NOT NULL THEN
        SELECT EXISTS(SELECT 1 FROM client WHERE id = p_client_id)
        INTO v_client_exists;
```

```
        IF NOT v_client_exists THEN
            p_message := 'Клиент не найден';
            ROLLBACK;
            RETURN;
        END IF;
    END IF;

    -- Блокировка и получение данных
    SELECT quantity, retail_price
    INTO v_current_stock, v_price
    FROM product
    WHERE id = p_product_id
    FOR UPDATE;

    -- Проверка остатка
    IF v_current_stock < p_quantity THEN
        p_message := 'Недостаточно товара. В наличии: ' || v_current_stock;
        ROLLBACK;
        RETURN;
    END IF;

    v_total_amount := v_price * p_quantity;

    -- Списываем товар
    UPDATE product
    SET quantity = quantity - p_quantity
    WHERE id = p_product_id;
```

```
    -- Создаём продажу
    INSERT INTO sale (
        number, date, employee_id, client_id,
        payment_type_id, total_amount
    ) VALUES (
        'SALE-' || TO_CHAR(NOW(), 'YYYYMMDD-HH24MISS'),
        NOW(),
        p_employee_id,
        p_client_id,
        p_payment_type_id,
        v_total_amount
    )
    RETURNING id INTO p_sale_id;

    -- Создаём позицию
    INSERT INTO sale_item (sale_id, product_id, quantity, price)
    VALUES (p_sale_id, p_product_id, p_quantity, v_price);

    -- Фиксируем
    COMMIT;

    p_success := TRUE;
    p_message := 'Продажа оформлена. ID: ' || p_sale_id;
END;
$$;
```

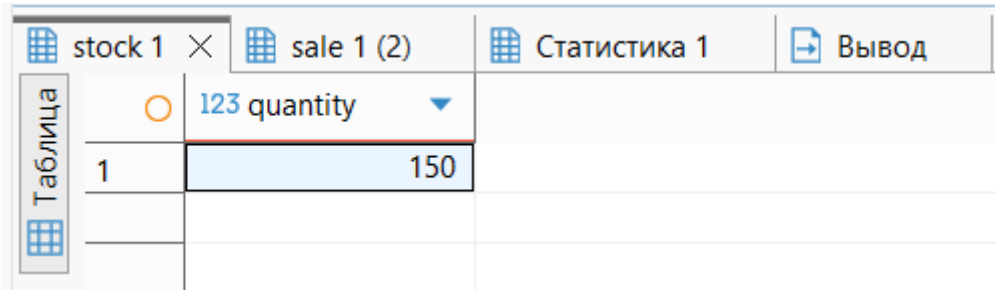
ЗАДАНИЕ 3. Демонстрация атомарной транзакции (COMMIT)

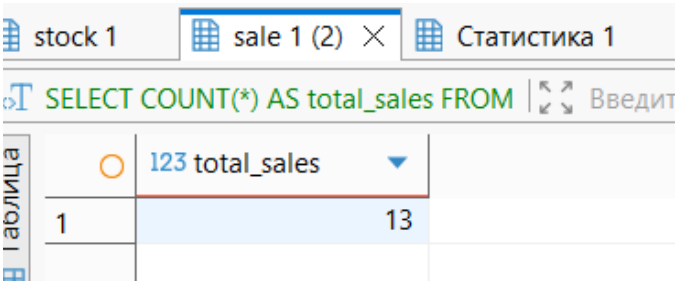
Задача: Показать, что две связанные операции (UPDATE + INSERT) выполняются атомарно — либо обе, либо ни одна.

Шаг 1. Фиксируем состояние «ДО»

```sql
-- Проверяем текущий остаток Coca-Cola в Главном магазине (store_id=1)
SELECT quantity FROM stock WHERE product_id = 1 AND store_id = 1;

-- Проверяем количество продаж
SELECT COUNT(*) AS total_sales FROM sale;
```





Шаг 2. Выполняем атомарную операцию внутри транзакции

```sql
BEGIN;

-- Операция 1: Списываем 5 шт. Coca-Cola из Главного магазина
UPDATE stock
SET quantity = quantity - 5
WHERE product_id = 1 AND store_id = 1;

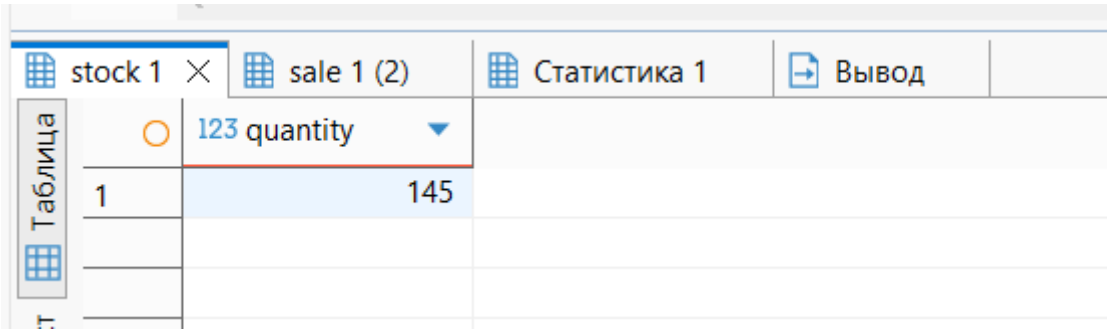
-- Операция 2: Создаём новую продажу на сумму 425 руб. (5 * 85)
INSERT INTO sale (
    number, date, employee_id, client_id,
    payment_type_id, total_amount
) VALUES (
    'TEST-ATOMIC-001',
    NOW(),
    1,          -- Смирнов А.В.
    1,          -- Иванов А.П.
    2,          -- Банковская карта
    425.00
);

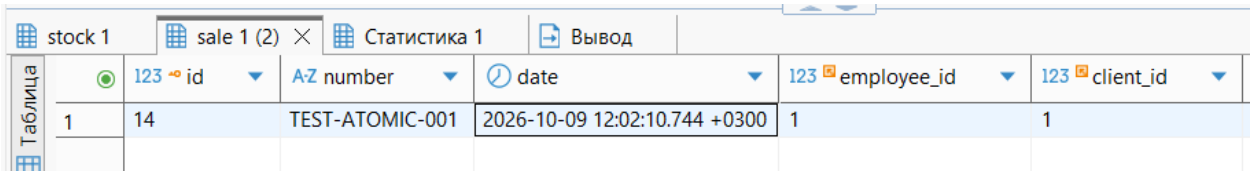
-- Фиксируем изменения навсегда
COMMIT;
```

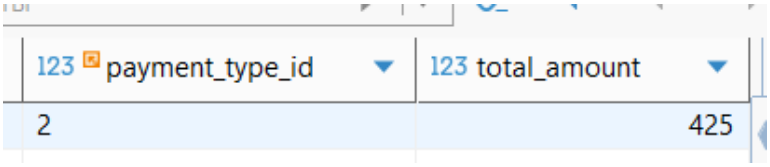
Шаг 3. Фиксируем состояние «ПОСЛЕ»

```
-- Проверяем, что остаток уменьшился ровно на 5
SELECT quantity FROM stock WHERE product_id = 1 AND store_id = 1;
-- Результат: 145 (было 150)

-- Проверяем, что продажа создалась
SELECT * FROM sale WHERE number = 'TEST-ATOMIC-001';
-- Результат: 1 строка с суммой 425.00
```





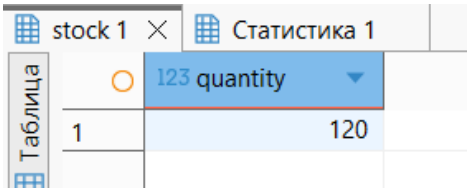


ЗАДАНИЕ 4. Демонстрация отката транзакции (ROLLBACK)

Задача: Смоделировать ошибку внутри транзакции и показать, что все изменения отменяются.

Шаг 1. Фиксируем исходное состояние

```
SELECT quantity FROM stock WHERE product_id = 2 AND store_id = 1;
-- Pepsi в Главном магазине. Ожидаемый результат: 120
```



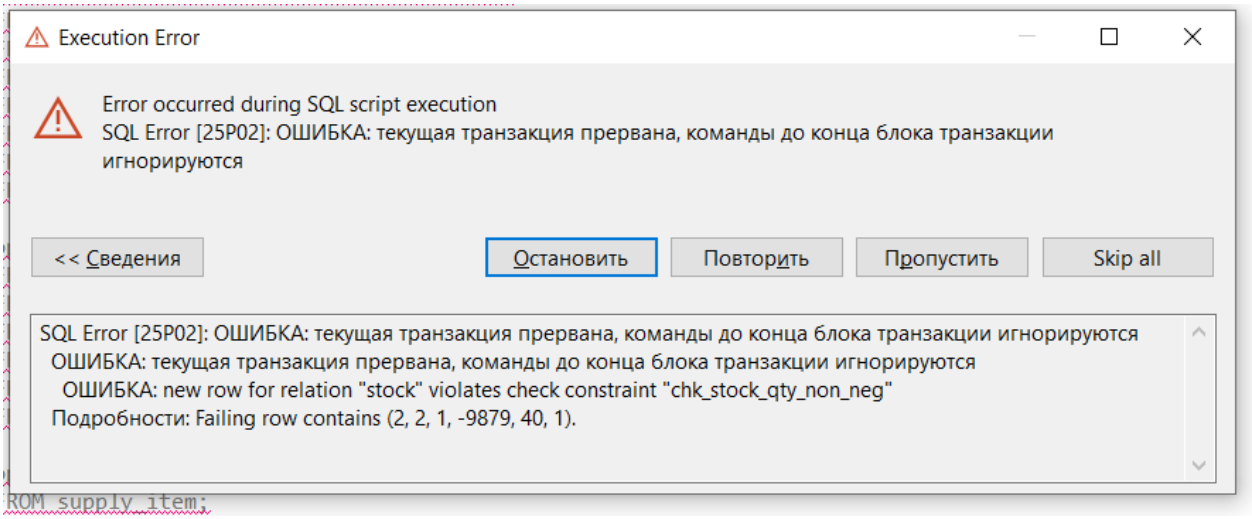
Шаг 2. Выполняем транзакцию с заведомой ошибкой

```
BEGIN;

-- Операция 1: Пытаемся списать 9999 шт. (явно больше, чем есть)
UPDATE stock
SET quantity = quantity - 9999
WHERE product_id = 2 AND store_id = 1;
-- Запрос выполнится без ошибки! PostgreSQL не проверяет CHECK при UPDATE внутри транзакции до COMMIT

-- Операция 2: Эта операция никогда не выполнится, т.к. транзакция уже "испорчена"
INSERT INTO sale (number, date, employee_id, client_id, payment_type_id, total_amount)
VALUES ('ERROR-SALE-001', NOW(), 1, 1, 2, 1000.00);

-- Попытка зафиксировать — НЕВОЗМОЖНА, транзакция в состоянии aborted
COMMIT;
```




```
ROM supply item;
```

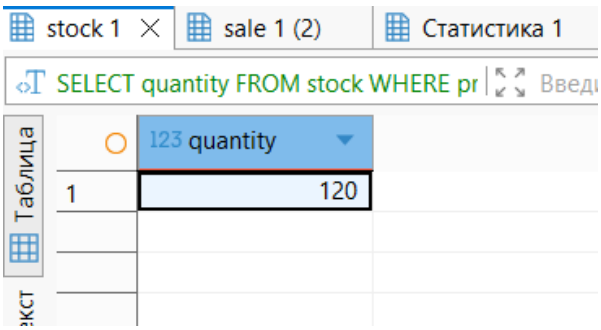
Шаг 3. Явный откат

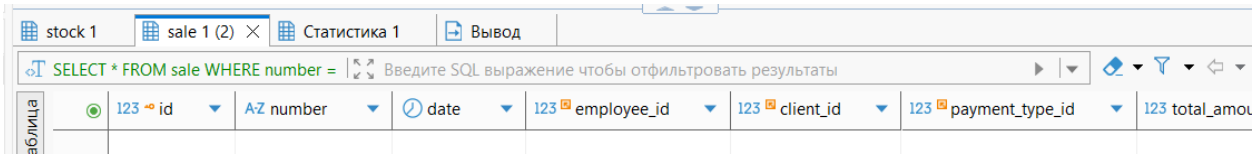
```
-- Выходим из состояния aborted
ROLLBACK;
```

Шаг 4. Проверяем, что данные НЕ изменились

```
-- Остаток должен остаться прежним!
SELECT quantity FROM stock WHERE product_id = 2 AND store_id = 1;


-- Продажа не должна существовать
SELECT * FROM sale WHERE number = 'ERROR-SALE-001';
```



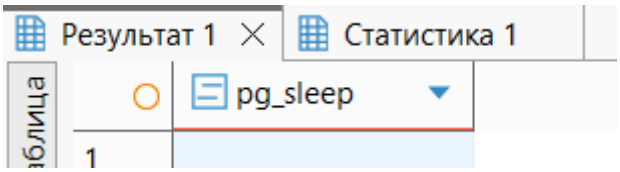


ЗАДАНИЕ №5. Моделирование трёх аномалий параллельного доступа

5.1 Аномалия «Грязное чтение» (Dirty Read)

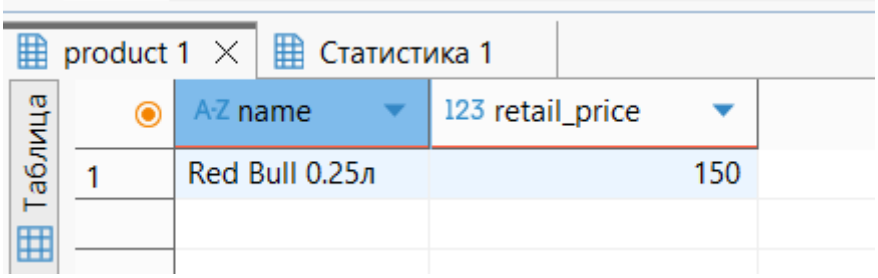
Окно А (Транзакция-изменитель):

```
BEGIN;
-- Изменяем цену Red Bull, но НЕ делаем COMMIT
UPDATE product SET retail_price = 999.99 WHERE id = 8;
-- Ждём 10 секунд, чтобы Окно Б успело прочитать
SELECT pg_sleep(10);
-- Откатываем изменение (цена должна вернуться к 150)
ROLLBACK;
```



Окно Б (Транзакция-читатель, запускается сразу после Окна А):

```
-- Читаем цену Red Bull во время сна Окна А
SELECT name, retail_price FROM product WHERE id = 8;
```



5.2 Аномалия «Неповторяемое чтение» (Non-Repeatable Read)

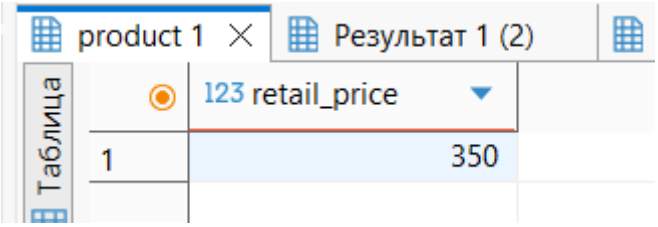
Задание: Одна транзакция читает одни и те же данные дважды и получает разные результаты, потому что другая транзакция успела изменить и зафиксировать их.

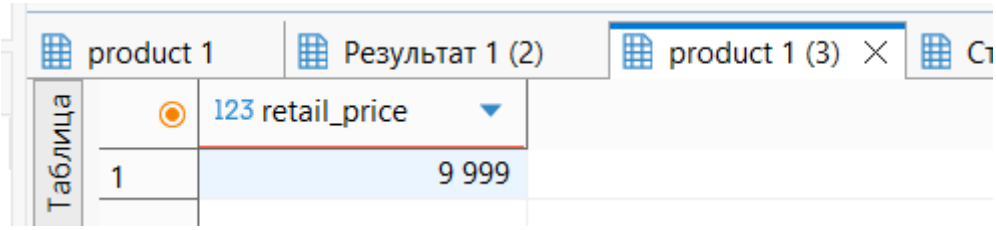
Окно А (Аудитор):

```
BEGIN;
-- Первое чтение цены Jacobs Кофе
SELECT retail_price FROM product WHERE id = 11;


-- Искусственная задержка 10 секунд
SELECT pg_sleep(10);

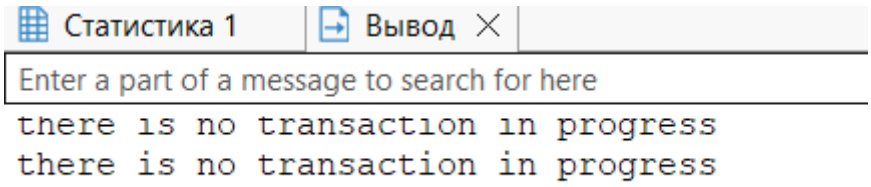
-- Второе чтение той же цены в ТОЙ ЖЕ транзакции
SELECT retail_price FROM product WHERE id = 11;
-- аномалия, цена изменилась между чтениями
COMMIT;
```





Окно Б (Менеджер, запускается во время pg_sleep):

```
-- Меняем цену и сразу фиксируем
UPDATE product SET retail_price = 9999.00 WHERE id = 11;
COMMIT;
```



Устранение аномалии через REPEATABLE READ

Окно А:

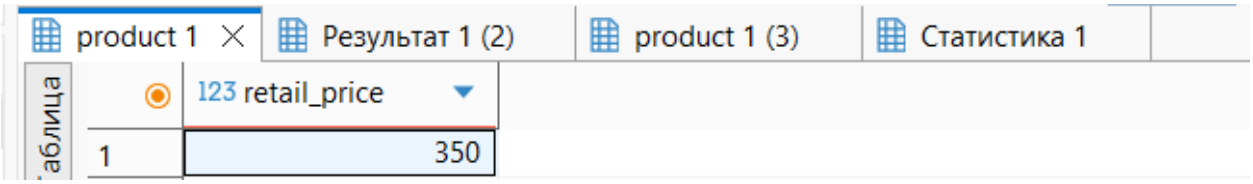
```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

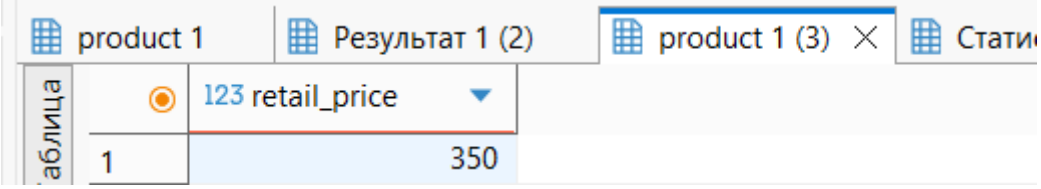
-- Первое чтение (данные на начало транзакции)
SELECT retail_price FROM product WHERE id = 11;

SELECT pg_sleep(10);

-- REPEATABLE READ НЕ увидит изменений, сделанных Окном Б
SELECT retail_price FROM product WHERE id = 11;

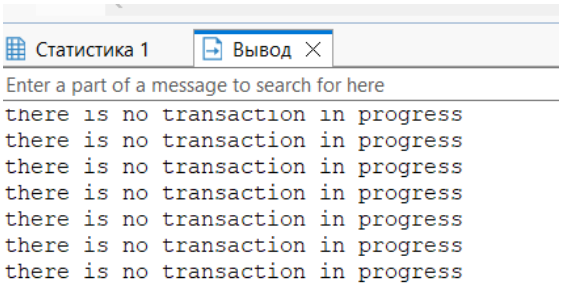
COMMIT;
```





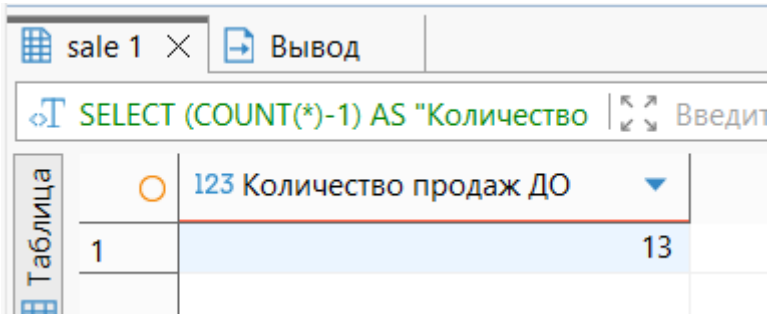
Окно Б:

```sql
-- Меняем цену на другое значение
UPDATE product
SET retail_price = 12000.00
WHERE id = 11;
COMMIT;
```



5.3 Аномалия «Фантомное чтение» (Phantom Read)

Задание: Появление новых строк («фантомов») в результатах агрегатной выборки внутри одной транзакции.



Окно А (Аналитик):

```sql
-- Окно А: Транзакция аналитика
BEGIN;

-- Первое чтение: считаем общее количество продаж
SELECT COUNT(*) AS "Количество продаж" FROM sale;

-- Искусственная задержка 10 секунд
SELECT pg_sleep(10);

-- Второе чтение: проверяем, изменилось ли количество
SELECT COUNT(*) AS "Количество продаж" FROM sale;

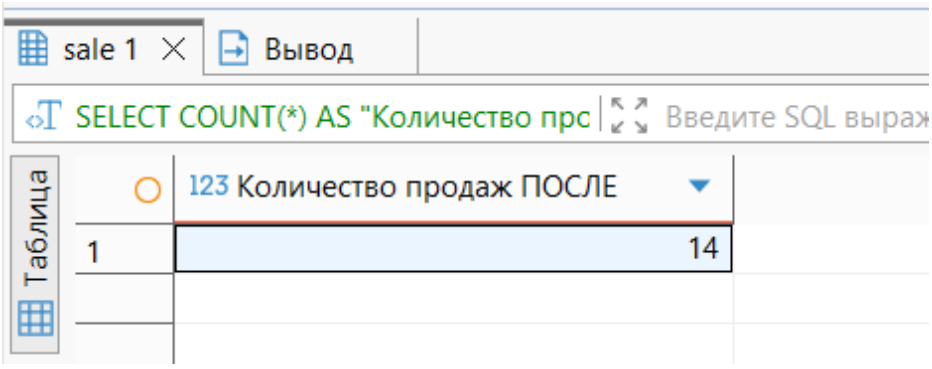
COMMIT;
```

Окно Б (Кассир, запускается во время pg_sleep):

```sql
-- Окно Б: Кассир добавляет новую продажу
BEGIN;

INSERT INTO sale (number, date, employee_id, client_id, payment_type_id, total_amount)
VALUES ('PHANTOM-SALE-999', NOW(), 1, 1, 2, 500.00);

COMMIT;
```



Устранение через SERIALIZABLE

Окно А (обновлённая версия):

```sql
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

SELECT COUNT(*) AS sales_count FROM sale;
-- РЕЗУЛЬТАТ 1: 13

SELECT pg_sleep(10);

-- Второе чтение — фантомов нет!
SELECT COUNT(*) AS sales_count FROM sale;
-- РЕЗУЛЬТАТ 2: 13 (аномалия устранена!)

COMMIT;
```

